

MISSION 4

Nous avons mis en place LVS (Linux Virtual Server), afin de garantir la disponibilité des services

LVS est un service virtuel composé de plusieurs serveurs, contrôlés par un directeur

Puisqu'il est composé de plusieurs serveurs, un LVS peut proposer plus de services, plus de capacités et de débit comparé à un serveur unique

Le directeur est totalement transparent, un client ne voit pas de LVS il croit être connecté à un serveur réel

On a donc de la redondance (on peut couper un serveur sans que le service s'arrête)

Keepalived est un moyen de contrôle des serveurs réels du LVS. Il permet de retirer du LVS un serveur réel dont le service serait arrêté (vérifier la disponibilité des services)

Les modifications se font dans `/etc/keepalived/keepalived.conf`

Avant de commencer nous devons respecter plusieurs conditions, le routage (NAT) doit être activé

Pour activer le routage (NAT) on modifie le fichier `/etc/sysctl.conf`, on enlève le # devant `net.ipv4.ip_forward=1`

La commande `sysctl -p` vérifie la configuration du fichier `/etc/sysctl.conf`

Le contexte a été respecté

La VIP côté client permettant d'accéder à la page est : 172,31,5,10

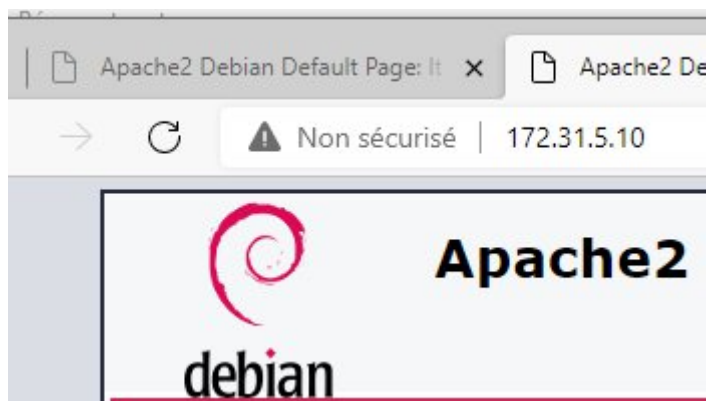
Le client lui possède une IP : 172,31,5,1 et une passerelle 172,31,5,254

Les adresses du Serveur réel UN et Deux sont : 192,168,2,100 et 192,168,2,101

La VIP GATEWAY côté serveur est : 192,168,2,120 (passerelle des deux serveurs)

Les serveurs réels écoutent sur le port 8001

<https://www.linux-fra.com/?p=2756> (synchronisation des serveurs)



```
P Virtual Server version 1.2.1 (size=4096)
rot LocalAddress:Port Scheduler Flags
  -> RemoteAddress:Port      Forward Weight ActiveConn InActConn
CP  172.31.5.10:80 rr
  -> 192.168.2.100:8001      Masq    1      0      1
  -> 192.168.2.101:8001      Masq    1      0      1
```

Concernant la synchronisation des données nous allons la faire via RSYNC, (synchronisation des données sous 60 secondes)

Nous devons donc installer les paquets suivants sur les DEUX serveurs réels : openssh-server, cron ainsi que rsync

On active le service ssh avec Systemctl start ssh ou systemctl enable —now ssh

Après l'avoir fait nous devons modifier le fichier avec « nano /etc/ssh/sshd_config »

Nous allons enlever l'# qui est sur « PermitRootLogin » et nous allons écrire juste à côté « yes » donc « PermitRootLogin yes » de cette manière la connexion SSH sera autorisée et un mot de passe ne sera pas demandé

Nous allons ensuite créer une clé avec cette commande « ssh-keygen -t rsa » sur le Serveur REEL UN

Cette clé sera ensuite envoyée sur le serveur réel deux, via cette commande ssh-copy-id root@192.168.2.101. (Toujours à réaliser sur le serveur Reel UN)

On pourra tester la connexion avec ssh root@192.168.2.101, la connexion doit être réalisable sans mot de passe

Avant de réaliser les commandes suivantes on sort de la connexion ssh en utilisant la commande « exit »

On teste manuellement rsync avec cette commande

```
rsync -avz --delete /var/www/html/ root@192.168.2.101:/var/www/html/
```

- **-a : Conserve les droits, les propriétaires et les dates des fichiers.**
- **-v : Affiche les fichiers en cours de transfert (mode verbeux).**

- **-z : Comprime les données pour économiser de la bande passante.**
- **--delete : Supprime les fichiers sur le serveur 2 s'ils ont été supprimés sur le serveur 1 (garantit l'intégrité).**

Nous allons ensuite configurer cron (automatisation)

On réalise la commande `crontab -e` (cela permet de mettre en place le planificateur)

On choisit l'éditeur de commande nano c'est le plus simple et répandu (il ne le demande qu'une fois)

Tout en bas du fichier on va mettre cette ligne (le nombre d'étoiles est importante)

```
* * * * * rsync -avz --delete /var/www/html/ root@192.168.2.101:/var/www/html/
```

(En mettant `* * * * *`, on dit à Linux : "Peu importe la minute, l'heure, le jour ou le mois, exécute la commande, le système vérifie le fichier chaque minute donc la synchronisation se lance toutes les 60 secondes)

L'utilisation de clés SSH permet une réplication sans intervention humaine, la transparence est donc assurée

La synchronisation n'est prise en compte que d'un seul côté, celui du serveur REEL Un au serveur REEL deux pas l'inverse, si on crée un fichier via le serveur réel deux, une minute plus tard ce fichier sera supprimé sur le serveur reel UN et serveur reel DEUX

C'est une réplication unidirectionnelle

(test après une minute)

```
root@ReelUn:~# touch /var/www/html/test_synchro.txt
root@ReelUn:~# ls /var/www/html/
```

```
root@ReelDeux:~# ls /var/www/html/
index.html test_synchro.txt
```